



Gymnázium J. K. Tyla

Úvod do agentického AI a harness pro automatizovaný softwarový vývoj

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D
Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Tato odborná práce se zabývá principy agentního inženýrství (*agentic engineering*) a architekturou řídicích harnessů pro automatizovaný vývoj softwaru. Praktickým přínosem práce je návrh a implementace systému DarkFactory — agentního harnessu instalovatelného jako aplikace pro platformu GitHub (GitHub App). Systém usiluje o maximální možnou míru automatizace vývojového cyklu od sémantické analýzy požadavků v GitHub Issues, přes technické plánování, až po generování kódu a vystavení pull requestu. Práce reflektuje, že současné agentní systémy nelze vnímat jako plně autonomní: jazykové modely vyžadují deterministické mantinely proti uvíznutí v nekonečných cyklech, správu kontextu bez sémantického posunu a především kontinuální zapojení člověka formou schvalovacích bran (*Human-in-the-loop*).

Klíčová slova

Agent (Agent), Agentní smyčka (Agent Loop), Chatbot (Chatbot), Degradace kontextu (Context Rot), Dovednosti (Skills), Git (Git), GitHub (GitHub), MCP (MCP), Promptové inženýrství (Prompt Engineering), Pull Request (Pull Request), Vektorová reprezentace (Embedding), Zapojení člověka do smyčky (Human-in-the-loop), Zásuvné moduly (Plugins), Řídící harness (Harness)

Agent (Agent)

Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí vícezkrokové akce směřující k dosažení zadaného cíle.

Agentní smyčka (Agent Loop)

Iterativní prováděcí cyklus autonomního agenta (založený na vzoru ReAct: Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí.

Chatbot (Chatbot)

Systém založený na jazykovém modelu určený k pasivní textové interakci s uživatelem; odpovídá na jednotlivé dotazy v chatu, avšak nedisponuje nástroji pro samostatnou modifikaci okolního prostředí.

Degradace kontextu (Context Rot)

Degradace pozornosti a kvality logického uvažování modelu způsobená zaplněním kontextového okna dlouhou historií a šumem, vedoucí k přehlížení instrukcí a ztrátě souvislostí.

Dovednosti (Skills)

Znovupoužitelné modulární balíčky instrukcí (SKILL.md), procedurálních pravidel a pomocných skriptů, které harness dynamicky načítá do kontextu agenta podle povahy řešeného úkolu.

Git (Git)

Distribuovaný systém správy verzí umožňující sledování historie změn kódu, větvení a deterministické vracení k předchozím funkčním stavům repozitáře.

GitHub (GitHub)

Cloudová platforma pro hosting gitových repozitářů, správu vývojového cyklu (Issues, Pull Requests) a automatizaci CI/CD pracovních postupů.

MCP (MCP)

Model Context Protocol — otevřený standard navržený společností Anthropic pro standardizovanou komunikaci mezi jazykovými modely a externími nástroji či datovými zdroji přes protokol JSON-RPC.

Promptové inženýrství (Prompt Engineering)

Inženýrská metodika systematického návrhu, strukturování a optimalizace instrukcí a systémových promptů pro řízení chování a mantinelů jazykového modelu.

Pull Request (Pull Request)

Formální návrh na začlenění změn z jedné větve repozitáře do druhé, který slouží jako platforma pro automatizované testování (CI), kódovou revizi člověkem a diskusi o navržených úpravách.

Vektorová reprezentace (Embedding)

Vícerozměrná vektorová reprezentace textu a tokenů, v níž geometrická vzdálenost a úhel vektorů zachycují sémantickou příbuznost a významové vztahy.

Zapojení člověka do smyčky (Human-in-the-loop)

Návrhový vzor vyžadující autorizaci lidského operátora formou schvalovacích bran (Human Gates) v klíčových rozhodovacích bodech před provedením nevratných systémových operací.

Zásuvné moduly (Plugins)

Zásuvné moduly běžící přímo v běhovém prostředí harnessu, které rozšiřují jeho exekuční jádro o specializované systémové adaptéry, ovladače nástrojů a deterministické záchytné body.

Řídicí harness (Harness)

Řídicí postroj — aplikační a orchestrační vrstva obklopující inferenční jádro modelu, která zajišťuje běhové prostředí nástrojů, dynamickou správu kontextového okna, bezpečnostní mantinely a deterministické řízení životního cyklu požadavku.

Annotation

This thesis explores the foundational principles of agentic engineering and harness architecture for automated software development. The practical contribution of the work is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize the automation of the software engineering lifecycle, ranging from issue interpretation in GitHub Issues and technical planning to code generation and pull request delivery. The thesis emphasizes that modern agentic systems cannot be considered fully autonomous: large language models require deterministic guardrails against infinite loops, rigorous context management without semantic drift, and indispensable human-in-the-loop governance via approval gates.

Keywords

Agent, Agent Loop, Chatbot, Context Rot, Embedding, Git, GitHub, Harness, Human-in-the-loop, MCP, Plugins, Prompt Engineering, Pull Request, Skills

Agent

Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí vícezkrokové akce směřující k dosažení zadaného cíle.

Agent Loop

Iterativní prováděcí cyklus autonomního agenta (založený na vzoru ReAct: Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí.

Chatbot

Systém založený na jazykovém modelu určený k pasivní textové interakci s uživatelem; odpovídá na jednotlivé dotazy v chatu, avšak nedisponuje nástroji pro samostatnou modifikaci okolního prostředí.

Context Rot

Degradace pozornosti a kvality logického uvažování modelu způsobená zaplněním kontextového okna dlouhou historií a šumem, vedoucí k přehlížení instrukcí a ztrátě souvislostí.

Embedding

Vícerozměrná vektorová reprezentace textu a tokenů, v níž geometrická vzdálenost a úhel vektorů zachycují sémantickou příbuznost a významové vztahy.

Git

Distribuovaný systém správy verzí umožňující sledování historie změn kódu, větvení a deterministické vracení k předchozím funkčním stavům repozitáře.

GitHub

Cloudová platforma pro hosting gitových repozitářů, správu vývojového cyklu (Issues, Pull Requests) a automatizaci CI/CD pracovních postupů.

Harness

Řídicí postroj — aplikační a orchestrační vrstva obklopující inferenční jádro modelu, která zajišťuje běhové prostředí nástrojů, dynamickou správu kontextového okna, bezpečnostní mantinely a deterministické řízení životního cyklu požadavku.

Human-in-the-loop

Návrhový vzor vyžadující autorizaci lidského operátora formou schvalovacích bran (Human Gates) v klíčových rozhodovacích bodech před provedením nevratných systémových operací.

MCP

Model Context Protocol — otevřený standard navržený společností Anthropic pro standardizovanou komunikaci mezi jazykovými modely a externími nástroji či datovými zdroji přes protokol JSON-RPC.

Plugins

Zásuvné moduly běžící přímo v běhovém prostředí harnessu, které rozšiřují jeho exekuční jádro o specializované systémové adaptéry, ovladače nástrojů a deterministické zachytňivé body.

Prompt Engineering

Inženýrská metodika systematického návrhu, strukturování a optimalizace instrukcí a systémových promptů pro řízení chování a mantinelů jazykového modelu.

Pull Request

Formální návrh na začlenění změn z jedné větve repozitáře do druhé, který slouží jako platforma pro automatizované testování (CI), kódovou revizi člověkem a diskusi o navržených úpravách.

Skills

Znovupoužitelné modulární balíčky instrukcí (SKILL.md), procedurálních pravidel a pomocných skriptů, které harness dynamicky načítá do kontextu agenta podle povahy řešeného úkolu.

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	4
Annotation	6
Keywords	7
1 Úvod	11
1.1 Motivace a vymezení problému	11
1.2 Cíl práce a výzkumné otázky	11
1.3 Metodika práce	11
2 Teoretická část – Analýza konceptu	12
2.1 Git a GitHub (Správa verzí)	12
2.1.1 Úvod	12
2.2 LLM, chatboti a agenti	12
2.2.1 Úvod	12
2.2.2 Tokeny, tokenizace a embedding	13
2.3 Harness a agentní inženýrství (prompt, kontext, smyčka, graf...)	13
2.3.1 Úvod	13
2.3.2 Agent vs. Chatbot	13
3 Praktická část – Návrh architektury	15
4 Výsledky a diskuse	16
5 Závěr	17
Seznam příloh	18
Seznam zdrojů	19

Seznam obrázků a tabulek

1 Úvod

1.1 Motivace a vymezení problému

1.2 Cíl práce a výzkumné otázky

Hlavní cíl: Vymezit teoretické principy agentního inženýrství (*agentic engineering*) a navrhnout modulární architekturu řídicího harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíčových rozhodovacích bodech.

Díličí cíle:

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu řídicího harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

1.3 Metodika práce

Práce má teoreticko-architektonický a inženýrský charakter. Vzhledem k dynamickému vývoji v oblasti autonomního softwarového vývoje práce důsledně zachovává a integruje zavedené anglické odborné názvy (např. *harness*, *pull request*, *agent loop*, *prompt engineering*, *skills* či *context rot*). Použití této terminologie je integrální součástí práce, neboť tyto anglické pojmy představují de facto celosvětové průmyslové standardy (*industry standards*), jejichž doslovný český překlad by byl nejednoznačný, zavádějící či v rozporu s běžnou inženýrskou praxí.

Postup práce sleduje strukturu inženýrského cyklu:

- **1. Analýza konceptu:** Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
- **2. Návrh architektury:** Formulace modulárního modelu řídicího harnessu, správy stavu, exekučního pískoviště, bezpečnostních pojistek a orchestrace subagentů.
- **3. Kritické zhodnocení:** Porovnání navržených principů s volnými agentními smyčkami a vymezení provozních limitů autonomního inženýrství.

2 Teoretická část – Analýza konceptu

2.1 Git a GitHub (Správa verzí)

2.1.1 Úvod

Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem. Jazykové modely generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém [Git](#) v kombinaci s platformou [GitHub](#).

Klíčové komponenty infrastruktury zahrnují:

- **Distribuovaný systém Git (1):** Ukládá kompletní historii projektu v podobě jednotlivých revizí (*commitů*). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.
- **Platforma GitHub:** Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - **Zadávání a sledování úkolů (Issues):** Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - **Revize změn (Pull Requests):** Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - **Automatizace (GitHub Actions):** Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

Agent v tomto pojetí nevystupuje jako černá skříňka s proprietárním protokolem, nýbrž jako standardní přispěvatel, který plně respektuje běžné vývojářské zvyklosti a nástroje.

2.2 LLM, chatboti a agenti

2.2.1 Úvod

V agentním softwarovém inženýrství vystupuje velký jazykový model (LLM) jako stochastické kognitivní jádro celého systému. Z hlediska vnitřní architektury se jedná o dekodérový transformer (*Decoder-only*), jehož typickými představiteli jsou moderní modely řad Claude, GPT či DeepSeek (2). Role modelu nespočívá ve vystupování jako vševědoucí orákulum se spolehlivou znalostí okolního světa, nýbrž jako pokročilý generátor hypotéz, kódu a strukturovaných volání nástrojů řízený obdrženým kontextem.

Základní principy fungování modelu zahrnují:

- **Autoregresivní predikce:** Model zpracovává zadanou sekvenci textu a na jejím základě iterativně předpovídá nejpravděpodobnější následující symboly (tokeny).

- **Stochastická povaha:** Vzhledem k pravděpodobnostnímu vzorkování může model na totožný vstup reagovat mírně odlišně, což vyžaduje deterministické mantinely v nadřazeném řídicím harnessu.

Pro efektivní nasazení modelu do vývojového cyklu je nezbytné porozumět způsobu, jakým reprezentuje informace a jaké fyzické limity vymezují jeho operační paměť.

2.2.2 Tokeny, tokenizace a embedding

Jazykový model nepracuje přímo se znaky ani slovy v lidském slova smyslu. Vstupní text je nejprve deterministickým algoritmem převeden na číselné reprezentace, se kterými následně počítají maticové vrstvy neuronové sítě.

Tento proces zahrnuje následující pojmy:

- **Tokeny a tokenizér:** Token představuje základní diskrétní jednotku (celé slovo, slabiku či fragment znaků). Převod mezi textem a posloupností číselných tokenů zajišťuje tokenizér (nejčastěji na bázi algoritmu Byte-Pair Encoding, BPE).
- **Embedding:** Každý token je promítnut do vícerozměrného vektorového prostoru, kde geometrická vzdálenost a úhel vektorů vyjadřují sémantickou příbuznost pojmů (např. vektorová analogie král – muž + žena $\approx$ královna).
- **Jazyková asymetrie tokenizace:** Vzhledem k trénovacím datům optimalizovaným primárně pro angličtinu spotřebovávají flektivní jazyky s bohatou diakritikou (včetně češtiny) 2× až 3× více tokenů pro vyjádření téhož významu.

Z inženýrského hlediska je proto žádoucí vést systémové prompty, technické plány i komunikaci mezi nástroji v angličtině, aby se šetřila kapacita kontextu a snížila latence inference.

2.3 Harness a agentní inženýrství (prompt, kontext, smyčka, graf...)

2.3.1 Úvod

V terminologii agentního inženýrství označuje pojem **Harness** aplikační vrstvu, která obklopuje samotné inferenční jádro jazykového modelu. Samotné inferenční jádro provádí výhradně matematické maticové operace nad zadanými váhami a vektory tokenů; veškerou orchestraci, práci se soubory a řízení bezpečnosti zajišťuje harness.

Ústřední komponentou a hlavní prováděcí funkcí, která v architektuře harnessu řídí samotný běh a iterativní koordinaci agenta v reálném vývojovém prostředí, je takzvaná **Agent Loop** (agentní smyčka).

2.3.2 Agent vs. Chatbot

Rozdíl mezi konverzačním **Chatbot**em a autonomním **Agent**em nespočívá v odlišném jazykovém modelu, ale v architektuře jeho zapojení do pracovního prostředí. Zatímco chatbot funguje pasivně jako textový rádce, agent vystupuje jako aktivní vykonavatel úkolů.

Srovnání obou přístupů:

- **Konverzační chatbot:**

- Reaguje pouze na přímé textové výzvy v uzavřeném okně chatu.
- Nemá přímý přístup k souborovému systému ani k nástrojům operačního systému.
- Uživatel musí navržený kód ručně zkopírovat, vložit do projektu a otestovat.
- **Autonomní agent:**
 - Je vybaven sadou výkonných nástrojů (*tools*) pro práci s repozitářem.
 - Aktivně prozkoumává soubory, modifikuje zdrojový kód, spouští testy a interpretuje jejich návratové kódy.
 - Funguje v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu vývojového prostředí.

3 Praktická část – Návrh architektury

4 Výsledky a diskuse

5 Závěr

Seznam příloh

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
2. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
3. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
4. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
5. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
6. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
7. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
8. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
9. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
10. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
11. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>